Ordering App: Technical Guide

Architecture, code, database and operations for developers and technical architects.

Version 1 · Last updated 4 October 2026

Contents

1. Architecture
2. Repository layout
3. Stack and versions
4. Setup, environment variables and new business setup
5. Database
6. Customer app internals
7. Admin panel internals
8. Order lifecycle
9. Partner API v1
10. Voice ordering
11. Build and deploy
12. Quality checks
13. Troubleshooting
14. Tech debt, switched-off features and dropped items

Who this guide is for

- **New developers:** to set up the project, find code quickly and make safe changes.
- **Technical architects:** to understand the design, the data model, the security model and the known risks.

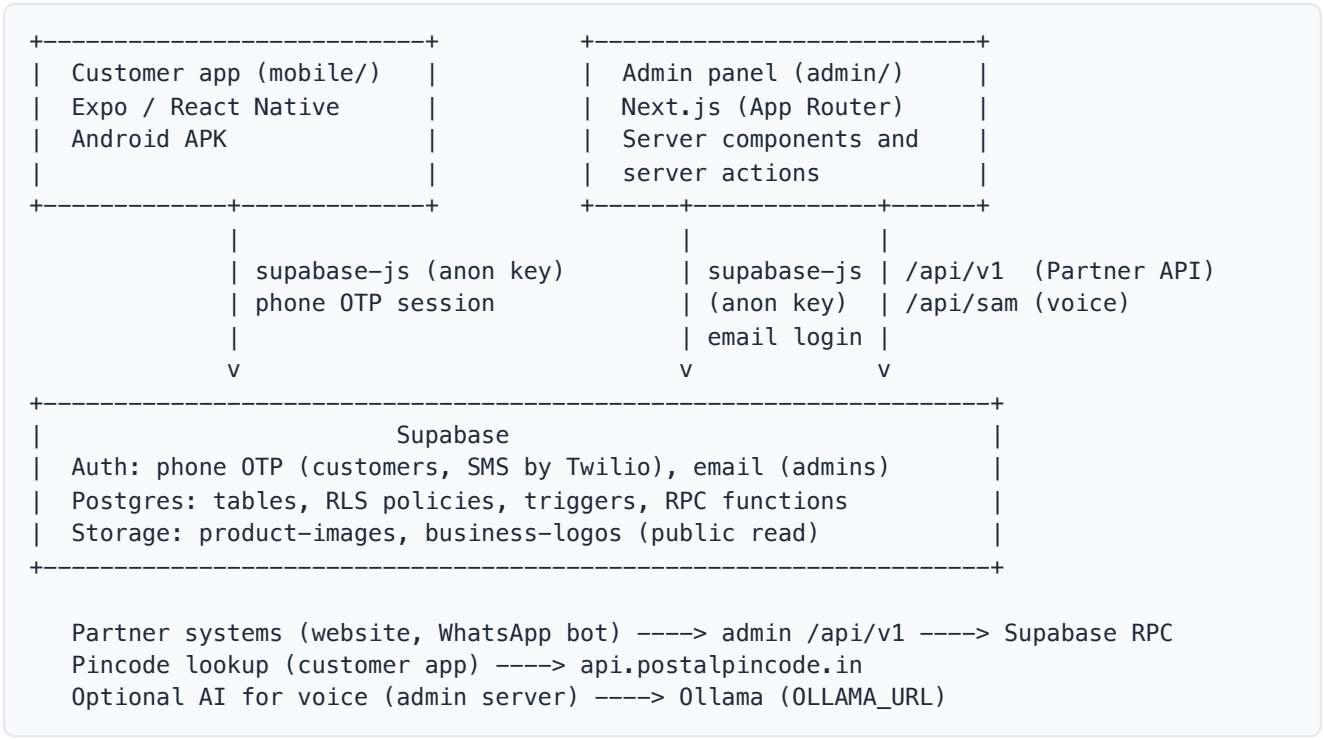
For screens and business rules in plain language, read the **Business Guide** first. This guide does not repeat them in detail.

Glossary

Term	Meaning
Business	One shop. Every row in the database belongs to one business (`business_id`).
Admin	A user whose `profiles.role = 'admin'` and who has a `business_id` .
Customer	A user who signed in with phone OTP. Has a `profiles` row and a `customers` row.
RLS	Row Level Security. Postgres rules that decide which rows each user can read or write.
RPC	A Postgres function called from the app with `supabase.rpc(...)` .
Security definer	A Postgres function that runs with the owner's rights, so it can bypass RLS in a controlled way.
Open order	Status `new` or `confirmed` .
Edit mode	The customer app state while modifying an open order.
Anon key	The public Supabase key. It is safe in apps because RLS protects data.
Service-role key	The secret Supabase key that bypasses RLS. **Never** used in either app.
`sam`	Folder name used in code for the voice and chat ordering assistant (`admin/lib/sam/` , `mobile/src/sam/`).

1. Architecture

Three parts share one Supabase project:



Key design decisions:

Decision	Reason
No custom backend server. Apps talk to Supabase directly.	Small team, low cost. Security is enforced by RLS and security-definer functions.
Only the anon key is used, in both apps.	A leaked app bundle must not expose full database access.
One business per deployment (`EXPO_PUBLIC_BUSINESS_ID`).	The schema is multi-business, but the customer app serves one shop.
Orders store a snapshot of customer name, mobile, address and item prices.	Later edits to products or addresses must not change past orders.
Order rules live in the database where possible (one open order per address, modify limit, order numbers, status history).	Rules then hold for every client: app, admin and partner API.
Product catalog is cached on the phone.	Fast start and browsing without internet.

2. Repository layout

```
order-app/
  package.json              Root scripts: admin, mobile, docs:pdf
  README.md
  docs/                     These guides, pending-and-future-scope.md, PDF output
  supabase/
    migrations/             35 SQL files, run in name order
    scripts/                Reusable SQL for support and testing (not migrations)
    documentation/          admin-setup.md, phone-otp.md
  mobile/                   Customer app (Expo)
    app.json, app.config.js, eas.json
    src/
      app/                  Expo Router screens (file = route)
        (shop)/             Tab group: home, buy-again, orders, cart, account, ...
        order/[id].tsx      Order details
        address/form.tsx    Add or edit an address
        confirmation.tsx, new-order.tsx, checkout.tsx, login.tsx, verify.tsx, sam.tsx
      cart/                 Cart state, edit mode, address book, submit and limits
      catalog/              ShopCatalogContext (products, categories, business)
      components/           UI components (ProductCard, ModifyOrderBar, OrderProgress, ...)
      services/             Supabase calls (orders, addresses, auth, products, ...)
      sam/                  Voice ordering client and NewOrderAddressPicker
      network/              NetworkProvider (online or offline)
      config/               env.ts, constants.ts, theme.ts
      types/, utils/
  admin/                    Admin panel (Next.js)
    proxy.ts                Session refresh and login redirect (replaces middleware)
    app/
      (admin)/              Dashboard, orders, products, customers, communities, reports, se
      api/v1/               Partner API routes
      api/sam/turn/         Voice ordering endpoint
      login/, unauthorized/
    components/             Client and server UI components
    services/               Data loading (orders.ts, ...) and server actions (*-actions.ts)
    services/supabase/      browser.ts, server.ts, admin-session.ts
    lib/                    partner-api.ts, partner-openapi.ts, sam/
    scripts/sam-eval.ts     Voice parser test script
    types/, utils/, config/
```

3. Stack and versions

Area	Technology	Version
Customer app	Expo SDK	~57.0.19
	React Native	0.86.3
	Expo Router	~57.0.18
	React	19.2.3

Area	Technology	Version
	TypeScript	~6.0.3
	AsyncStorage	for session, cart and catalog cache
Admin panel	Next.js (App Router, `proxy.ts`)	16.3.4
	React	19.2.8
	Tailwind CSS	4
	`@supabase/ssr` and `@supabase/supabase-js`	supabase-js 2.114
Backend	Supabase (Postgres, Auth, Storage)	hosted
SMS	Twilio (through Supabase phone auth)	
Builds	EAS Build (Android)	CLI >= 16.17.0
Docs	md-to-pdf	latest through `npx`

Important: Expo 57 and Next.js 16 changed many APIs. Read the versioned Expo docs at https://docs.expo.dev/versions/v57.0.0/ and the Next.js 16 docs before writing code. In Next.js 16, `middleware.ts` is replaced by `proxy.ts`.

4. Setup and environment variables

4.1 First-time setup

```
cd mobile && npm install
cd ../admin && npm install
cp mobile/.env.example mobile/.env
cp admin/.env.example admin/.env
```

Then:

1. Create a Supabase project. Run every file in `supabase/migrations/` in the SQL editor, **in name order** (see 5.6).
2. Create the business, the admin auth user and the admin profile. Follow `supabase/documentation/admin-setup.md`.
3. Set up phone OTP with Twilio. Follow `supabase/documentation/phone-otp.md`. For testing, add a test phone number and fixed OTP in Supabase Auth (format `91XXXXXXXXXX=123456`), so no real SMS is sent.
4. Fill in the `.env` files (below).

4.2 Run

What	Command	Notes
Admin panel	`npm run admin` (root) or `cd admin && npm run dev`	http://localhost:3000, sign in at `/login`
Customer app	`npm run mobile` (root) or `cd mobile && npx expo start`	Open in Expo Go or a development build
PDF guides	`npm run docs:pdf` (root)	Writes `docs/pdf/*.pdf`

Without Supabase variables, the admin panel runs with mock data and a local admin. This is for UI work only.

4.3 Environment variables

Customer app (`mobile/.env`, and the EAS `preview` environment for builds):

Variable	Required	Purpose
`EXPO_PUBLIC_SUPABASE_URL`	Yes	Supabase project URL
`EXPO_PUBLIC_SUPABASE_ANON_KEY`	Yes	Supabase anon key
`EXPO_PUBLIC_BUSINESS_ID`	Yes	The shop's `businesses.id`
`EXPO_PUBLIC_SAM_API_URL`	No	Voice endpoint, for example `https://<admin-host>/api/sam/turn`
`EXPO_PUBLIC_VOICE_ORDERING`	No	`true` shows voice ordering. Leave unset in this phase. Missing from `.env.example`.

All `EXPO_PUBLIC_*` values are built into the app and are public. Never put secrets there.

4.4 Setting up for a new business

The product is not tied to one shop. Business name, logo, banner, description, products, categories and prices come from the database and are set in the admin panel. A few values are still fixed in code and must be changed for each new business:

What	Where
App name, slug, URL scheme, Android package, voice permission texts	`mobile/app.json`

What	Where
App icon, splash and fallback logo on the welcome screen	`mobile/assets/images/`, `mobile/src/components/WelcomeSignup.tsx`
Business id	`EXPO_PUBLIC_BUSINESS_ID` (in `.env` and the EAS environment)
Storage key prefix	see 6.3
Admin sidebar and login logo and name	`admin/components/AdminShell.tsx`, `admin/app/login/page.tsx`, `admin/public/branding/`
Voice assistant replies and AI prompt (mention the shop name)	`admin/lib/sam/replies.ts`, `admin/lib/sam/intent-prompt.ts`, `mobile/src/sam/useSamSpeech.ts`
Package names	`mobile/package.json`, root `package.json`

Then create the business row, the admin user and the admin profile (`supabase/documentation/admin-setup.md`), and set the logo and banner in **Settings > Branding**.

Moving these values into environment variables or the `businesses` table is listed as tech debt in section 14.

Admin panel (`admin/.env`):

Variable	Required	Purpose
`NEXT_PUBLIC_SUPABASE_URL`	Yes	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Yes	Supabase anon key
`OLLAMA_URL`	No	Local AI server for voice ordering, for example `http://127.0.0.1:11434`
`OLLAMA_MODEL`	No	AI model name. Default `llama3.2`.

5. Database

5.1 Tables

Table	Purpose	Key columns
`businesses`	One row per shop	`business_name`, `logo_url`, `banner_url`, `phone`, `email`, `address`, `description`, `communities_enabled`, `categories_enabled`

Table	Purpose	Key columns
`profiles`	One row per auth user	`id` (= `auth.users.id`), `role` (`customer` or `admin`), `business_id`, `name`, `mobile`, `email`
`customers`	Shop's customer list (app and offline)	`name`, `mobile`, `mobile_key` (last 10 digits, unique per business), `source` (`app`, `whatsapp`, `walk_in`, `phone`, `other`), `notes`, `auth_user_id`
`customer_addresses`	Address book, max 3 per customer	`user_id` or `customer_id`, `label` (`home`, `work`, `other`), `name`, `mobile`, `house`, `locality`, `address`, `pincode`, `city`, `instructions`, `is_default`, `community_id`, `community_name`
`products`	Catalog	`name`, `description`, `ingredients`, `price`, `image_url`, `available` (false = Coming soon), `active` (false = hidden), `category_id`
`product_categories`	Optional categories	`name` (unique per business), `image_url`, `active`
`orders`	One row per order	see 5.2
`order_items`	Lines of an order (snapshot)	`product_id`, `product_name`, `product_price`, `quantity`, `subtotal`
`order_status_events`	Status history with times	`order_id`, `status`, `note`, `created_at`
`order_counters`	Next order number per business	used by `assign_order_number`
`order_channels`	Allowed channel values with labels	`id`, `label`
`api_keys`	Partner API keys	`key_prefix`, `key_hash` (SHA-256), `channel`, `scopes`, `active`, `revoked_at`, `last_used_at`
`communities`	Optional address groups	`name`, `active`
`delivery_runs`, `delivery_run_orders`	Planned delivery batches	**Not used by either app yet**

Table	Purpose	Key columns
`product_info_requests`	Questions the voice assistant could not answer	for future product content

5.2 The `orders` table

Column	Values and meaning
`order_number`	100001 to 999999, per business. Set by trigger.
`status`	`new` (shown as Placed), `confirmed`, `out_for_delivery`, `delivered`, `cancelled`
`payment_method`	`pending`, `cash`, `upi`
`refund_status`	`null`, `pending`, `refunded`
`cancel_reason`	Text. Customer cancels always save "Customer requested" (set on the server).
`cancelled_by`	`customer` when the customer cancelled, otherwise `null`. Cleared by trigger whenever the status is not `cancelled`. Used for the daily cancel limit.
`channel`	`app`, `web`, `whatsapp`, `phone`, `walk_in` (Direct), `admin`, `api` (Partner API)
`external_id`	Partner's own order id. Unique per business and channel (idempotency).
`user_id`, `customer_id`	Auth user and `customers` row
`customer_address_id`	The saved address used. Drives the one-open-order rule.
`customer_name`, `mobile`, `delivery_address`, `pincode`	Snapshot at order time
`community_id`, `community_name`	Snapshot from the address (trigger)
`total_amount`	Sum of item subtotals
`placed_via`	`app` or `voice`
`update_count`	Number of customer modifications (max 5)

5.3 Security (RLS)

Every table has RLS on. The pattern is:

- **Customer rules:** `user_id = auth.uid()`. Customers can read their own profile, addresses, orders, order items and (since migration 34) their own order status events. They can insert orders for themselves, and items only into the order they placed in the last 15 minutes (still `new`, never modified). Later changes go through `customer_replace_order_items`.
- **Server-side prices (migration 36):** the app's prices and totals are never trusted. Triggers re-price every customer item from `products` (must be available and active) and recompute `total_amount`. On a customer order insert, `payment_method`, `refund_status`, `channel`, `external_id`, `cancel_reason`, `update_count`, `created_at` and `customer_id` are set by the server, and `customer_address_id` must be one of the customer's own addresses.
- **Admin rules:** `business_id = current_admin_business_id()`. Admins can read and change rows of their own business only.
- **Public shop data** (products, categories, business details) is read through security-definer RPCs, so the shop works before sign-in:
  - `get_available_products(p_business_id)`: active products
  - `get_shop_categories(p_business_id)`: active categories, when enabled
  - `get_business_public(p_business_id)`: name, logo, banner, description
- **Profile protection:** the `profiles_protect_privileges` trigger stops users from changing their own `role` or `business_id`, and keeps a customer's `mobile` equal to the phone verified at OTP sign-in.
- **Customer record:** customers cannot read the `customers` table (it holds the shop's notes). `ensure_customer_profile` links the record by the verified phone from the login token, not by a number sent by the app.
- **Storage:** buckets `product-images` and `business-logos` are public to read. Only admins can write. Uploads are limited to JPEG, PNG, GIF and WebP up to 5 MB.
- **Function access:** Supabase lets `anon` run new functions by default, so internal functions are revoked from `anon` explicitly. `partner_auth` and `partner_order_payload` run only inside the other partner functions.

Helper functions: `current_admin_business_id()`, `current_customer_business_id()`, `is_admin()`.

5.4 Triggers

Trigger	Table	What it does
`*_set_updated_at`	most tables	Keeps `updated_at` current

Trigger	Table	What it does
`profiles_protect_privileges`	`profiles`	Blocks role and business changes by users; keeps the customer's account phone equal to the verified login phone
`orders_customer_guard`	`orders`	On customer inserts, sets payment, refund, channel and bookkeeping fields on the server; checks the address belongs to the customer
`order_items_customer_guard`	`order_items`	Re-prices customer items from `products` (available and active only)
`order_items_customer_totals`	`order_items`	Recomputes the order total after a customer adds items
`customer_addresses_owner_guard`	`customer_addresses`	Customers can link an address only to their own customer record
`orders_assign_order_number`	`orders`	Sets the next 6-digit order number
`orders_log_status`	`orders`	Writes `order_status_events` on insert ("Order placed") and on each status change ("Updated by admin" or the cancel reason)
`orders_snapshot_community`	`orders`	Copies community from the address
`customer_addresses_before_write`	`customer_addresses`	Fills `business_id` and `customer_id`, enforces the 3-address limit
`customer_addresses_sync_default`	`customer_addresses`	When one address becomes default, clears the others
`customer_addresses_protect_community`	`customer_addresses`	Only admins can set or change the community. A new customer address copies the community only when all the customer's mapped addresses share one.
`products_category_same_business`	`products`	Category must belong to the same business

Trigger	Table	What it does
`orders_reset_cancelled_by`	`orders`	Clears `cancelled_by` when the status is not `cancelled`

5.5 Functions (RPC)

Function	Called by	Purpose
`ensure_customer_profile`	Customer app after OTP	Creates or updates the `profiles` row and the `customers` row (source `app`), using the verified phone from the login token
`get_available_products`, `get_shop_categories`, `get_business_public`	Customer app	Public shop data
`find_open_customer_order`	Customer app	Finds the open order for an address
`customer_cancel_order`	Customer app	Cancels the customer's own order only while `new`, at most 3 customer cancellations in 24 hours (a per-customer lock stops parallel cancels from beating the limit). Ignores the reason sent by the app and saves "Customer requested".
`customer_replace_order_items`	Customer app	Replaces the items of an open order in one transaction. Checks owner, status, 1 to 15 lines, 1 to 10 each, the 5-modification limit, and that the address belongs to the customer. Optional `p_expected_items_revision` raises `ORDER_STALE` on conflict. Prices come from shared SQL with the admin path. Writes an `order_modifications` audit row, increments `items_revision` and `update_count`.
`admin_replace_order_items`	Admin panel	Same open-order item rules as the customer replace RPC (status `new` or `confirmed`, server-side pricing). Optional revision check. Writes `order_modifications` with `actor_type = admin`. Increments `items_revision` only (does not touch customer `update_count`, payment, or refund fields).

Function	Called by	Purpose
`partner_auth`, `partner_shop`, `partner_products`, `partner_product`, `partner_place_order`, `partner_get_order`, `partner_list_orders`, `partner_order_payload`	Admin `/api/v1` routes	Partner API (see section 9)
`ensure_order_channels`	Migrations	Seeds channel values

5.6 Migrations

Run every file in `supabase/migrations/` in name order, in the Supabase SQL editor. The last files end with `notify pgrst, 'reload schema';` so the API sees changes at once.

#	File (date prefix removed)	What it adds or changes
1	`init`	Core tables, RLS, storage buckets, shop RPCs
2	`ensure_customer_profile`	Profile creation after OTP sign-in
3	`order_number`	6-digit order numbers per business
4	`reload_order_number_api`	Schema reload for the API
5	`order_number_idempotent`	Safe re-run of the order number setup
6	`business_branding`	`banner_url`
7	`order_channels_partner_api`	Channels, `external_id`, `api_keys`, partner functions
8	`customer_addresses`	Address book with default address
9	`order_payment_method`	`payment_method`
10	`order_status_workflow`	`cancel_reason`
11	`order_status_events`	Status history table and trigger, backfill
12	`order_refund_status`	`refund_status`
13	`admin_customers`	`customers` table, admin address management, `other` label
14	`order_channels_sources`	More channels and customer sources
15	`communities`	Communities and (unused) delivery runs
16	`community_address_mapping`	Map addresses to communities

#	File (date prefix removed)	What it adds or changes
17	`communities_enabled`	On or off switch
18	`order_community_from_address`	Order community snapshot
19	`product_categories`	Categories and `categories_enabled`
20	`product_category_same_business`	Business check on categories
21	`shop_categories_rpc`	`get_shop_categories`
22	`shop_categories_visible`	Only active categories
23	`category_images`	Category `image_url`
24	`product_active`	`products.active`
25	`sam_ingredients_feedback`	`ingredients`, `product_info_requests`
26	`open_order_per_address`	`customer_address_id`, unique open-order index, find, cancel and replace RPCs
27	`replace_order_address`	Replace RPC can also change the address
28	`orders_lookup_indexes`	Faster order lookups
29	`remove_preparing_status`	Moves `preparing` to `confirmed`, adds `out_for_delivery`
30	`address_city`	`customer_addresses.city`
31	`fix_address_community_trigger`	Fixes "function max(uuid) does not exist"
32	`order_placed_via_update_limit`	`placed_via`, `update_count`, 5-modification limit
33	`order_update_limit_message`	Friendlier limit message
34	`order_status_events_customer_read`	Customers can read their own status history
35	`customer_cancel_rules`	`cancelled_by`; customers cancel only while `new`, max 3 per 24 hours; status-aware modify limit message
36	`security_hardening`	Server-side prices and totals, locked customer order fields, verified-phone customer linking, no customer read of `customers`, address and item rules, cancel lock, partner `active` check and input caps, function revokes, image bucket limits

#	File (date prefix removed)	What it adds or changes
37	`order_modifications`	`orders.items_revision`, `order_modifications` audit table, shared pricing helpers, `admin_replace_order_items`, customer replace extended with audit + optimistic locking, RLS for customers and admins to read history

Note: `supabase/documentation/admin-setup.md` lists migrations only up to `admin_customers`. Use the folder as the source of truth.

6. Customer app internals

6.1 Navigation (Expo Router)

```
_layout.tsx  (providers: Network, ShopCatalog, Cart)
  index.tsx              Entry: decides welcome, login or home
  login.tsx, verify.tsx  Phone OTP
  (shop)/  tabs
     home                Visible tab
     buy-again           Visible tab
     orders              Visible tab (My orders)
     cart                Hidden tab (stays mounted, opened by View cart)
     account             Hidden tab
     categories          Hidden (Phase 2)
     reorder             Hidden, redirects
  checkout.tsx           "Saved addresses"
  address/form.tsx       Add or edit address
  new-order.tsx          Choose address when an order is already open
  confirmation.tsx       Order status after place, update or cancel
  order/[id].tsx         Order details
  product/[id].tsx       Product details (not linked in this phase)
  sam.tsx                Voice ordering (hidden; redirects to Home)
```

6.2 State and providers

Provider or module	File	Holds
`NetworkProvider`	`src/network/NetworkProvider.tsx`	Online or offline flag. `requireConnection` guards OTP and order actions.
`ShopCatalogContext`	`src/catalog/ShopCatalogContext.tsx`	Products, categories and business, loaded from cache first and then refreshed

Provider or module	File	Holds
`CartContext`	`src/cart/CartContext.tsx`	Cart items, edit mode, open order for the selected address
Address book	`src/cart/address-book.ts`	Saved addresses and the selected one (shared store with subscribe)

6.3 Saved data on the phone (AsyncStorage)

Key	Content
Supabase session	Managed by supabase-js
`<app>.cart.v1`	Cart items
`<app>.cart.edit.v1`	Edit mode: `{ orderId, fingerprint, items }` (original items)
`<app>.delivery-draft`	Delivery details draft
`<app>.entered-shop`	Whether the customer has passed the welcome screen
`<app>.catalog.products.v4`	Product cache
`<app>.catalog.business.v2`	Business cache
`<app>.catalog.categories.v3`	Category cache

`<app>` is the app's short name prefix, set in `src/cart/cart-storage.ts`, `src/cart/delivery-draft.ts`, `src/nav/shop-entry.ts` and `src/services/catalog-cache.ts`. When changing a cached data shape, increase the version number in the key.

6.4 Cart and edit mode

`CartContext` is the most complex module. Main fields:

Field	Meaning
`items`	Current cart lines
`editingOrderId`	Set while modifying an order
`editingItemsRevision`	`items_revision` captured when modify mode starts; sent as `p_expected_items_revision` on update
`originalItems`	Items as last saved on the server (for Discard and "Was N" tags)
`savedFingerprint`	Fingerprint of `originalItems`, to detect changes

Field	Meaning
`openOrder`	Open order for the selected address, if any
`openOrderReady`	`true` once the open-order lookup for the current address has finished
`rowChange(productId)`	Returns `{kind:'new'}`, `{kind:'changed', from}` or `null`
`canRevertEdit`, `revertEditChanges()`	Discard support

Flow to modify an order (`src/cart/modify-flow.ts`, `goModifyOrder`):

```
selectAddressForOrder(order)        switch to the order's address
  -> beginEditOrder(id, items, order)  load items, set openOrder
  -> navigate('/cart', { focus: 'add' })  cart scrolls to "Add more items"
  -> Update order -> customer_replace_order_items RPC (optional revision token)
  -> navigate to confirmation, then clearCart()
```

Shop-side item edits: admins change lines on the order detail page while the order is still Placed or Confirmed (`admin_replace_order_items`). Each successful replace appends a row to `order_modifications`. The customer app loads the latest unacknowledged admin change and shows a **Shop modified** tag until they tap **View changes** (ack stored in `AsyncStorage` per order and modification id).

Order status push: when status becomes **Confirmed**, **Out for delivery**, or **Delivered**, the Edge Function `send-order-status-push` sends Expo push to tokens in `push_tokens`. The admin server invokes it after `updateOrderStatus` when `SUPABASE_SERVICE_ROLE_KEY` is set. Setup: `supabase/documentation/push-notifications.md`.

Rules in code:

- Edit state and cart are saved together, and cleared together on restore, so the app never reopens a plain cart behind an open order.
- If the saved edit has no changes at restart, it is dropped.
- If the order is no longer open (shop moved it on, or cancelled it), changes are discarded with an alert.
- An 8-second lock stops double taps on Update order.
- The cart screen redirects to `/new-order?next=cart` when the address has an open order and the cart is not in edit mode. It waits for `openOrderReady` and shows a spinner, so there is no flash.

Limits: `src/cart/limits.ts`: 15 lines, 10 per line. The replace RPC checks them again on the server.

6.5 Placing an order (`src/services/orders.ts`)

1. Check: signed in, business set, cart within limits.
2. Insert into `orders` with `status = 'new'`, the address snapshot, `customer_address_id`, `community_id` and `total_amount` calculated in the app.
3. Insert the `order_items`.
4. A unique-index error (`23505`) means the address already has an open order. The app shows the New order screen.
5. If the network drops after insert, the app finds the just-placed order (`findJustPlacedOrder`) instead of placing a second one.

Timeouts: 15 seconds for placing, 4 seconds for lookups. All Supabase requests abort after 20 seconds (`src/services/supabase.ts`).

6.6 Addresses

- `src/services/addresses.ts` reads and writes `customer_addresses`. `setRemoteDefaultAddress(id)` sets one default; the trigger clears the rest.
- `src/cart/address-book.ts` switches the address at once (optimistic update), saves in the background and reloads if the save fails.
- City and state come from `api.postalpincode.in` by pincode.

6.7 Order tracking

`src/components/OrderProgress.tsx` draws Placed, Confirmed, Out for delivery and Delivered, with the time of each step from `order_status_events` (`getMyOrderById` loads them). If the events cannot be read (migration 34 not run), only the placed time shows.

6.8 App configuration

- `app.json`: app name, slug, URL scheme, Android package (`com.<business>.app`) and `versionCode`.
- `app.config.js`: when `EXPO_PUBLIC_VOICE_ORDERING` is not `true`, removes the speech plugin and blocks the `RECORD_AUDIO` permission.
- `expo-location` and `react-native-maps` are installed but not used (map entry was dropped).

7. Admin panel internals

7.1 Request flow

```
Browser -> proxy.ts           refreshes the Supabase session cookie;
                              redirects to /login if not signed in
        -> (admin)/layout.tsx  getAdminSession(): role = admin and business_id required,
                              else /unauthorized
        -> page (server component) loads data with services/*.ts
        -> client component calls a server action (services/*-actions.ts)
              each action checks the admin session again, then writes with RLS
```

- `proxy.ts` skips `/api/*`. Partner API and voice routes do their own checks.
- `getAdminSession` (`services/supabase/admin-session.ts`) is cached per request.
- Server action body limit is 6 MB (`next.config.ts`), for image uploads.
- All "today" and date filters use IST (`utils/ist-dates.ts`).

7.2 Main modules

Area	Pages	Components	Services
Dashboard	`(admin)/page.tsx`		`orders.ts`, `report-metrics.ts`
Orders	`orders/`, `orders/[id]`, `orders/new`, `customers/[id]/orders/new`	`OrdersBoard`, `OrderWorkflow`, `OrderStepper`, `OrderPaymentForm`, `AdminOrderForm`	`orders.ts`, `order-actions.ts`
Products	`products/`, `products/new`, `products/[id]`	`ProductsBoard`, `ProductForm`, `ProductImport`, `ProductCatalogMenu`, `AvailabilityToggle`	`products.ts`, `product-actions.ts`, `storage.ts`
Categories	`products/categories`	`CategoriesBoard`, `CategoryForm`	`categories.ts`, `category-actions.ts`
Customers	`customers/...`	`CustomersBoard`, `CustomerForm`, `CustomerAddressBook`	`customers.ts`, `customer-actions.ts`

Area	Pages	Components	Services
Communities	`customers/communities/...`	`CommunitiesBoard`, `CommunityCustomerManager`	`communities.ts`, `community-actions.ts`
Reports	`reports/`	`ReportsFilters`	`reports.ts`, `report-metrics.ts`
Settings	`settings/business-info`, `branding`, `api`	`BusinessInfoForm`, `BrandingForm`, `ApiKeysForm`	`business.ts`, `api-key-actions.ts`

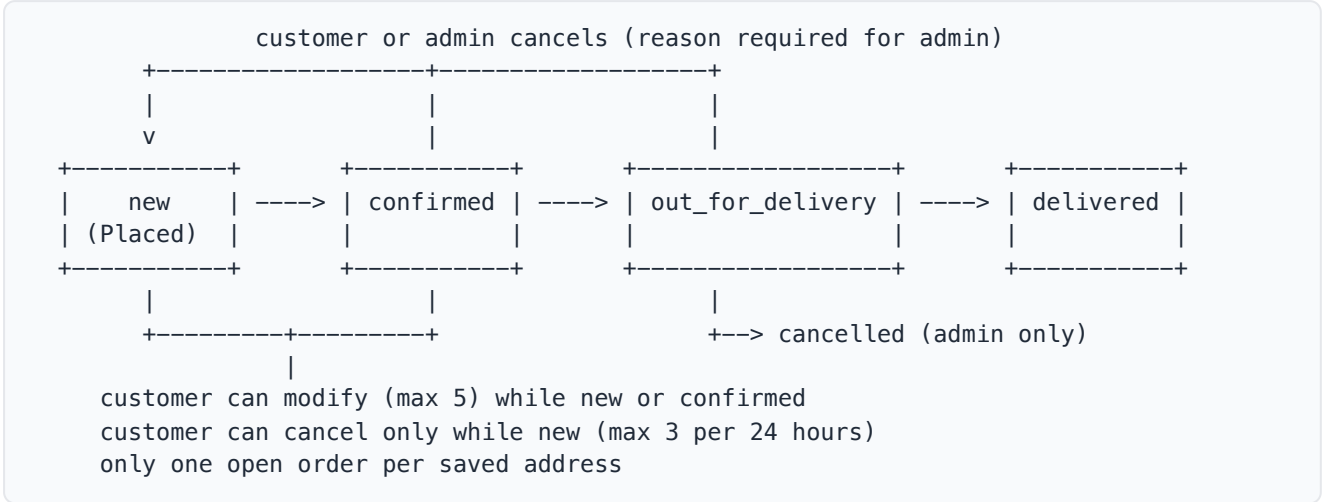
7.3 Feature switches in code

Switch	File	Effect
`SHOW_LATER_PHASE_MENUS = false`	`components/AdminShell.tsx`	Hides Settings from the menu
`LIST_QUICK_ACTIONS = false`	`components/OrdersBoard.tsx`	Hides next-step buttons on the orders list

7.4 Order actions (`services/order-actions.ts`)

- **Status change:** sets the new status. Cancelling a paid order sets `refund_status = 'pending'`. The trigger writes the status event.
- **Payment:** `cash`, `upi` or `pending` (Not yet). Refund: `pending` to `refunded`.
- **Correction mode** ("Fix a status mistake") sets any status directly.
- **Admin-created orders** (`services/customer-actions.ts`) use channel `phone` and prices read from `products` on the server.
- **Community mapping** (`services/community-actions.ts`) also updates the customer's existing orders for the same pincode.

8. Order lifecycle



Status	Customer can modify	Customer can cancel	Admin next action
`new`	Yes	Yes (max 3 per 24 hours)	Confirm order
`confirmed`	Yes	No (call the shop)	Mark out for delivery
`out_for_delivery`	No	No	Mark as delivered (asks for payment)
`delivered`	No	No	Record payment if outstanding
`cancelled`	No	No	Mark refunded if refund is pending

Payment states shown in the apps:

```
Not paid:          Awaiting payment --(delivered)--> Outstanding --> Paid (Cash/UPI)
Paid then cancel:  Paid --> Refund pending --> Refunded
Never paid, cancel: Not Applicable
```

Database guarantees:

- `orders_one_open_per_address`: unique index on `customer_address_id` where the status is open.
- `customer_replace_order_items`: only the owner, only open orders, at most 5 times.
- `customer_cancel_order`: only the owner, only while `new`, at most 3 customer cancellations in 24 hours (counted from `order_status_events` where `cancelled_by = 'customer'`).
- Admin actions use the admin RLS policies, not these functions, so the customer limits never apply to the admin panel.

- In the app, `canCancelOrder(status)` in `mobile/src/types/order.ts` hides Cancel for confirmed orders; the database enforces the same rule.
- `orders_log_status`: every status change gets a time-stamped event.

9. Partner API v1

Base path: `https://<admin-host>/api/v1`. Full OpenAPI document: `GET /api/v1/openapi`.

Authentication: send the key as `Authorization: Bearer qk_live_...` or `X-Api-Key: qk_live_...`. Keys are created in **Settings > API**. Only the SHA-256 hash is stored. Each key has a channel (`api` or `whatsapp`) and the scopes `catalog.read`, `orders.read`, `orders.write`.

Method	Path	Purpose
GET	`/api/v1`	API info
GET	`/api/v1/openapi`	OpenAPI document
GET	`/api/v1/shop`	Business details
GET	`/api/v1/products`	Available products
GET	`/api/v1/products/{id}`	One product
GET	`/api/v1/orders`	Orders placed with this key's channel
POST	`/api/v1/orders`	Place an order
GET	`/api/v1/orders/{id}`	One order

Place order body:

```
{
  "customer": { "name": "Priya", "mobile": "9876543210",
                "address": "12, Anna Nagar, Chennai", "pincode": "600040" },
  "items": [ { "product_id": "<uuid>", "quantity": 2 } ],
  "external_id": "web-1001"
}
```

- Prices come from the database, not from the request.
- `external_id` makes the call idempotent: sending the same id again returns the same order.
- The route (`lib/partner-api.ts`) uses the anon key and calls `partner_*` security-definer functions, which check the key hash.
- CORS is open (`*`), so browsers can call it. There is no rate limit yet.

10. Voice ordering

Status: hidden in this phase (see section 14).

```
Customer app (sam.tsx, src/sam/)
  speech-to-text on the phone
     |  POST { transcript, businessId, draft }
     v
Admin /api/sam/turn  -> lib/sam/process-turn.ts
     |  normalise input (English, Tamil, mixed)
     |  intent: Ollama AI if OLLAMA_URL is set, else rule-based fallback parser
     |  match products, update the draft cart, build the reply
     v
Reply { text, draft, ... }  -> app speaks the reply and shows product cards
```

- Main files: `admin/lib/sam/*` (parser, intents, matching, replies), `mobile/src/sam/*` (client, speech, session).
- Languages: `en`, `ta`, `mix`.
- Unknown product questions are saved in `product_info_requests`.
- Test the parser with `cd admin && npm run sam:eval`.
- Orders placed by voice get `placed_via = 'voice'`.

11. Build and deploy

11.1 Customer app (Android, EAS)

Profile	Output	Use
`development`	APK with dev client	Local development
`preview`	APK	Testing and current distribution
`production`	App bundle (AAB)	Google Play

Steps:

1. Increase `android.versionCode` in `mobile/app.json`.
2. Make sure the EAS `preview` environment has the three required `EXPO_PUBLIC_*` values. Leave `EXPO_PUBLIC_VOICE_ORDERING` unset.
3. From `mobile/`: `npx eas-cli build -p android --profile preview`.

11.2 Admin panel

Hosting is to be decided. There is no hosting configuration in the repo yet. Any Node.js host that supports Next.js 16 works (for example Vercel). Requirements:

- Set `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`.
- Use HTTPS (session cookies are marked secure behind HTTPS).
- Fix the known type errors first, because `next build` may fail on them (section 14).
- The CSV import reads image files from the server's disk. On a cloud host this works only for files on that host.

11.3 Database changes

1. Add a new file in `supabase/migrations/` named `YYYYMMDDHHMMSS_description.sql`.
2. Make it safe to run twice (`if not exists`, `create or replace`).
3. End with `notify pgrst, 'reload schema';` when it changes functions or columns.
4. Run it in the Supabase SQL editor. Add it to the go-live list in `docs/pending-and-future-scope.md` until it has run in production.

12. Quality checks

There are no automated tests yet. Before each change, run:

Check	Command
Customer app types	`cd mobile && npx tsc --noEmit`
Customer app lint	`cd mobile && npm run lint`
Admin types	`cd admin && npx tsc --noEmit`
Admin lint	`cd admin && npx eslint .`
Admin build	`cd admin && npm run build`
Voice parser	`cd admin && npm run sam:eval`

Known existing warnings: three admin type errors (`services/reports.ts`, `services/storage.ts`) and some `react-hooks/set-state-in-effect` lint errors (mobile `CartContext`, `address/form`, `sam.tsx`, `ShopCatalogContext`, `AddressPickerSheet`; admin `OrdersBoard`).

Manual test list for orders: place, modify (add, change, remove, Discard, Exit), the 5-modification limit, cancel, two addresses with two open orders, admin status steps with times, payment and

refund, offline banner, app restart while modifying.

13. Troubleshooting

Problem	Likely cause	Fix
"function max(uuid) does not exist" when adding an address	Migration 31 not run	Run `fix_address_community_trigger`
Saved addresses fail to load or save	Migration 30 not run (`city` missing)	Run `address_city`
App shows only the Placed time	Migration 34 not run	Run `order_status_events_customer_read`
"Could not find the function ..." from the API	Schema cache is old	Run `notify pgrst, 'reload schema';`
OTP SMS not received	Twilio setup, DLT template or sender id	Check `supabase/documentation/phone-otp.md`. Use the test number for development.
"Wait about a minute, then request a new OTP."	Supabase OTP rate limit	Wait, or use the test number
Admin shows "Not authorized"	Profile is not `role = 'admin'` or has no `business_id`	Fix the profile row (see `admin-setup.md`)
Admin shows mock data	Supabase variables missing in `admin/.env`	Set them and restart
App shows no products	Wrong `EXPO_PUBLIC_BUSINESS_ID`, or no active products	Check the id and product flags
Voice says it cannot connect	`EXPO_PUBLIC_SAM_API_URL` points to a local IP	Expected in this phase
"Modify limit reached"	`update_count` reached 5	Business rule. Customer can still cancel if the order is still `new`.

Problem	Likely cause	Fix
"Cancel limit reached for today"	Customer cancelled 3 orders in 24 hours	Business rule. The shop can cancel from the admin panel. To reset it for testing or support, run `supabase/scripts/reset-customer-cancel-limit.sql`.
"This order is confirmed, so it can no longer be cancelled"	Customers cancel only while `new`	Business rule. Customer can modify, or the shop can cancel.
Changes in `.env` have no effect in the app	Expo caches env values	Restart with `npx expo start -c`

14. Tech debt, switched-off features and dropped items

The live list, including future ideas, is `docs/pending-and-future-scope.md`. Update that file when anything changes. This section lists only what exists in the code today.

14.1 Must do before go-live

- Run migrations 27 to 35 in production (listed in the pending file).
- Build a new APK with the latest app changes.
- Decide and set up admin hosting.

14.2 Tech debt and risks

Item	Risk	Suggested fix
Customer order prices come from the app. Placing an order inserts `total_amount` and item prices sent by the app. `customer_replace_order_items` also trusts the prices in `p_items`.	A modified client could place orders at wrong prices. Low risk for a small shop that checks orders, but real.	Move placing into a security-definer RPC that reads prices from `products`, and do the same in the replace RPC.
Order and items are inserted in two calls, not one transaction	An order without items if the second call fails (the app retries and recovers, but not atomically)	Same RPC as above

Item	Risk	Suggested fix
Old SQL functions still list `preparing` and do not list `out_for_delivery`	Harmless after migration 29, but confusing	Clean up in a new migration
`customer_addresses_one_community` index is not created by any migration	Schema drift between environments	Add a migration or remove references
Partner functions check `available` but not `active`	Hidden products could be ordered by a partner	Add `active = true` checks
No Partner API rate limit; CORS open	Misuse with a leaked key	Add rate limits; restrict CORS
Admin type errors in `reports.ts` and `storage.ts`	`next build` may fail	Fix before hosting
Lint errors (`set-state-in-effect`)	Extra renders	Refactor when touching those files
No automated tests	Regressions	Start with order-flow tests on the RPCs
`OrderTimeline.tsx` unused; `admin/README.md` is boilerplate	Noise	Remove or rewrite
`expo-location` , `react-native-maps` unused	Bigger APK	Remove
`EXPO_PUBLIC_VOICE_ORDERING` missing from `mobile/.env.example`	Setup confusion	Add it with a comment
`admin-setup.md` migration list is out of date	Setup confusion	Point it to the migrations folder
Development timing logs in `mobile/src/services/supabase.ts`	Only in dev builds	Remove when performance is settled
`delivery_runs` tables unused	Dead schema	Build the feature or drop the tables
Reports read up to 3,000 orders in the server	Slow for large ranges	Move totals into SQL
Shop name and logo fixed in code in a few places (section 4.4)	Extra work to set up a new business	Read them from environment variables or the `businesses` table

14.3 Switched off in the code

- Voice ordering: needs a publicly hosted voice server and `EXPO_PUBLIC_VOICE_ORDERING=true`.
- Product detail page: pass `onPress` to `ProductCard` in Home and Buy again.
- Quick status actions on the orders list: `LIST_QUICK_ACTIONS = true`.
- Settings in the admin menu: `SHOW_LATER_PHASE_MENUS = true`.
- Decision needed: allow a new order to an address whose order is out for delivery?

14.4 Dropped

Item	Reason
`preparing` status	No value. Migrated to `confirmed`.
Wake word ("Hey ...")	Always-on listening: battery, data, privacy, Android 13+ only. If it returns, use an on-device engine such as Picovoice Porcupine.
Map-based address entry	Unreliable maps and search. Replaced by typed address with pincode lookup.
Call and WhatsApp items in the order menu	Simplified. If they return, build the WhatsApp link from the last 10 digits plus `91`.